

ՀՀ ԿԳՄՍ ՆԱԽԱՐԱՐՈՒԹՅՈՒՆ
ՀԱՅԱՍՏԱՆԻ ԵՎՐՈՊԱԿԱՆ ՀԱՄԱԼՍԱՐԱՆ
ՏԵՂԵԿԱՏՎԱԿԱՆ ՏԵԽՆՈԼՈԳԻԱՆԵՐ ԵՎ ԿԻՐԱՌԱԿԱՆ
ՄԱԹԵՄԱՏԻԿԱԿԱՆ ԱՄԲԻՈՆ

ԿՈՒՐՍԱՅԻՆ ԱՇԽԱՏԱՆՔ

IRONPULSE ՖԻԹՆԵՑ ՀԱՎԵԼՎԱԾ

ՄԱՍՆԱԳԻՏՈՒԹՅՈՒՆ: «Տեղեկատվական տեխնոլոգիաներ»

ԴԱՍԸՆԹԱՑ: Ծրագրավորում ժամանակակից հարթակներում 2

ԿՈՒՐՍ: III առկա

ՈՒՍԱՆՈՂ: Վալերի Խաչատրյան

ԴԱՍԱԽՈՍ: Ս. Պետրոսյան

գիտական աստիճան, կոչում ԱԱՀ

ստորագրություն

Բովանդակություն

Ներածություն.....	3
Գլուխ 1 Համակարգի ճարտարապետություն	5
1.1 ճարտարապետության ընդհանուր պատկերը	5
1.2. Տվյալների բազայի նախագծումը և արդյունավետության օպտիմալացումը	6
1.3. Անվտանգության ապահովում, վավերացում և սխալների կենտրոնացված մշակում.....	7
ԳԼՈՒԽ 2. API ՎԵՐՋՆԱԿԵՏԵՐԻ ՖՈՒՆԿՑԻՈՆԱԼՈՒԹՅՈՒՆԸ ԵՎ FRONTEND ԻՐԱԿԱՆԱՑՈՒՄԸ	10
2.1. Մարզումների վերջնակետերի գործառույթները և սերվերային վերլուծությունը (Analytics)	10
2.2. Թեթև և առանց ֆրեյմվորքի առաջնամասի (Frontend) իրականացումը.....	11
ԵԶՐԱԿԱՑՈՒԹՅՈՒՆ	13
Օգտագործված գրականության ցանկ.....	15

Ներածություն

IronPulse-ը ֆիթնես մարզումների հետևման լիարժեք (full-stack) վեբ հավելված է, որը նախագծված է ֆիթնեսի սիրահարների շրջանում տարածված մի խնդիր լուծելու համար. այն է՝ ֆիզիկական ակտիվությունը գրանցելու, կալորիաների ծախսը վերահսկելու և ժամանակի ընթացքում մարզումների առաջընթացը տեսանկարելու (վիզուալիզացնելու) կենտրոնացված, կառուցվածքային և ծրագրավորողի համար թափանցիկ գործիքի բացակայությունը: Գոյություն ունեցող բազմաթիվ ֆիթնես հավելվածներ կամ վճարովի են, կամ ծանրաբեռնված են ավելորդ գործառնություններով, կամ էլ օգտատիրոջ տվյալներին ծրագրային հասանելիություն չեն տրամադրում: IronPulse-ը ստեղծվել է այս խնդիրը լուծելու համար՝ համատեղելով մաքուր, ժամանակակից օգտատիրոջ ինտերֆեյսը (UI) լավ նախագծված REST API հետնամասի (backend) հետ, որը ծրագրավորողները կարող են հասկանալ, ընդլայնել և ինքնուրույն տեղակայել (deploy):

Հավելվածը թույլ է տալիս օգտատերերին ստեղծել հաշիվ, ապահով կերպով մուտք գործել համակարգ և անմիջապես սկսել իրենց մարզումների գրանցումը տարբեր կատեգորիաներում, ներառյալ՝ Կարդիո (Cardio), Ուժային (Strength), Յոգա (Yoga), HIIT, Պիլատես (Pilates), Քրոսֆիթ (CrossFit), Լող (Swimming), Հեծանվավազք (Cycling), Վազք (Running) և Ձգումներ (Stretching): Մարզման յուրաքանչյուր գրառում ֆիքսում է պարապմունքի անվանումը, կատեգորիան, տևողությունը բոլորներով, այրված կալորիաները, ամսաթիվը և ցանկության դեպքում՝ անձնական նշումներ: Ժամանակի ընթացքում այս տվյալները վերածվում են իմաստալից վիճակագրության՝ ամբողջ ժամանակահատվածի հանրագումարների, շաբաթական վերլուծությունների և ըստ կատեգորիաների բաշխումների, ինչն օգտատերերին տալիս է իրենց մարզումների սովորությունների հստակ ու անկեղծ պատկերը:

Բացի հետևումից, IronPulse-ը ներառում է Սննդառության խորհուրդների բաժին, որը կրթում է օգտատերերին այն դիետիկ սկզբունքների մասին, որոնք նպաստում են մարզական արդյունավետությանը և վերականգնմանը: Ուղեցույցը ներառում է մակրոնուտրիենտներ (մակրոսննդանյութեր), ինչպիսիք են ցածր յուղայնությամբ սպիտակուցները, բարդ ածխաջրերը և առողջ ճարպերը, ինչպես նաև հիդրատացիայի (ջրային հաշվեկշռի)

ռազմավարությունները, հակաօքսիդանտներով հարուստ մթերքները և մարզումից հետո վերականգնողական սնունդը: Կից մարզագույքի ուղեցույցը բացատրում է սովորական մարզասրահի գործիքների դերը, ներառյալ՝ ծանրաձողերը (barbells), համտելները (dumbbells), վազքուղիները (treadmills), հեծանվամարզասարքերը (stationary bikes), դիմադրության ժապավենները (resistance bands) և թիավարման սարքերը (rowing machines)՝ օգնելով օգտատերերին հասկանալ, թե ինչպես կազմակերպել արդյունավետ մարզումներ՝ անկախ իրենց փորձառության մակարդակից:

Նախագիծն ամբողջությամբ կառուցված է բաց կոդով (open-source) տեխնոլոգիաներով, չի պահանջում որևէ վճարովի ծառայություն՝ բացառությամբ MongoDB-ի տվյալների բազայի, և կարող է տեղայնորեն (locally) գործարկվել հինգ րոպեից պակաս ժամանակում: Առաջնամասը (frontend) չի պահանջում կոդի հավաքման փուլ (build step), փաթեթների կառավարիչ (package manager) կամ ֆրեյմվորք. այն մեկ HTML ֆայլ է, որն անմիջապես բացվում է ցանկացած ժամանակակից բրաուզերում:

Գլուխ 1 Համակարգի ճարտարապետություն

1.1 Ճարտարապետության ընդհանուր պատկերը

IronPulse-ի backend-ը կառուցված է **Node.js**-ի վրա, որը JavaScript-ի գործարկման միջավայր է (runtime environment) և օգտագործում է իրադարձությունների վրա հիմնված (event-driven), չարգելափակող (non-blocking) I/O մոդել: Այն հատկապես հարմար է մասշտաբավորվող ցանցային հավելվածների և API-ների կառուցման համար: Node.js-ը թույլ է տալիս ամբողջ հավելվածը (թե՛ սերվերային, թե՛ հաճախորդի տրամաբանությունը) գրել JavaScript-ով՝ նվազեցնելով կոնտեքստի փոփոխությունը (context switching) և հնարավորություն տալով կոդի համատեղ օգտագործմանը: Node.js-ի հիմքի վրա հավելվածն օգտագործում է **Express.js**-ը՝ թեթև և ոչ կանոնակարգող վեբ ֆրեյմվորք, որն ապահովում է երթուղավորում (routing), middleware-ների համադրում և HTTP օժանդակ մեթոդներ՝ առանց նախագծի կոշտ կառուցվածք պարտադրելու:

Նախագիծը հետևում է **Model-View-Controller (MVC)** ճարտարապետական մոդելին՝ առանձնացնելով անելիքները հստակ սահմանված շերտերի միջև: `models` դիրեկտորիան (թղթապանակը) պարունակում է Mongoose սխեմաներ, որոնք սահմանում են MongoDB-ում պահվող տվյալների ձևն ու սահմանափակումները: `controllers` դիրեկտորիան պարունակում է բիզնես տրամաբանությունը՝ ֆունկցիաներ, որոնք մշակում են ստացված հարցումները, փոխազդում են տվյալների բազայի հետ և վերադարձնում պատասխաններ: `routes` դիրեկտորիան քարտեզագրում է HTTP մեթոդները և URL հասցեները դեպի համապատասխան controller ֆունկցիաներ: `middleware` դիրեկտորիան պարունակում է հարցումների մշակման բազմակի օգտագործման ֆունկցիաներ, որոնք գործարկվում են նախքան controller-ները՝ կատարելով վավերացման ստուգում (authentication), մուտքային տվյալների ստուգում (validation) և սխալների ձևաչափում: `utils` դիրեկտորիան պարունակում է փոքր, կենտրոնացված օժանդակ (helper) ֆունկցիաներ, որոնք օգտագործվում են ամբողջ հավելվածում, ներառյալ JWT թոքենների գեներատորը և պատասխանների ստանդարտացված ձևաչափիչը (formatter):

1.2. Տվյալների բազայի նախագծումը և արդյունավետության օպտիմալացումը

MongoDB-ն ընտրվել է որպես IronPulse-ի տվյալների բազա՝ իր փաստաթղթերի վրա հիմնված (document-oriented) բնույթի պատճառով, ինչը բնականորեն համապատասխանում է API-ում օգտագործվող JSON տվյալների ձևաչափին: Ի տարբերություն ռելյացիոն տվյալների բազաների, MongoDB-ն չի պահանջում աղյուսակի ֆիքսված սխեմա՝ ապահովելով ճկունություն մշակման ընթացքում, մինչև ժամանակ թույլ տալով հավելվածի շերտում Mongoose-ի միջոցով կիրառել տվյալների խիստ վավերացման կանոններ: **Mongoose**-ն ավելացնում է սխեմայի սահմանման շերտ MongoDB-ի բնիկ դրայվերի վրա՝ թույլ տալով ծրագրավորողներին սահմանել դաշտերի տեսակները, պարտադիր պայմանները (required), նվազագույն և առավելագույն արժեքները, թվարկված տարբերակները (enum), լռելյայն արժեքները (default) և սխալների անհատականացված հաղորդագրությունները: `User` մոդելը սահմանում է դաշտեր օգտատիրոջ անվան (username), էլ. փոստի, գաղտնաբառի, տարիքի, քաշի և հասակի համար, ընդ որում՝ ժամանակային նշումները (timestamps) ավտոմատ կերպով կառավարվում են Mongoose-ի կողմից: `Workout` մոդելը սահմանում է դաշտեր տվյալ օգտատիրոջ համար (որպես MongoDB ObjectId հղում), մարզման անվան, կատեգորիայի, տևողության, այրված կալորիաների, ամսաթվի և նշումների համար՝ նույնպես ավտոմատ ժամանակային նշումներով:

Տվյալների բազայի արդյունավետությունը (performance) ապահովվում է `Workout` սխեմայի վրա սահմանված բաղադրյալ ինդեքսների (compound indexes) միջոցով: Օգտատիրոջ ID-ի և ամսաթվի համակցության վրա ստեղծված ինդեքսը թույլ է տալիս ամենատարածված հարցումը՝ օգտատիրոջ մարզումների ստացումը ըստ ամսաթվի տեսակավորված, կատարել առանց ամբողջ հավաքածուի սկանավորման (full collection scan): Օգտատիրոջ ID-ի և կատեգորիայի վրա հիմնված երկրորդ ինդեքսը արագացնում է գտված (filtered) հարցումները: Մարզման անվան դաշտի վրա ստեղծված տեքստային ինդեքսը (text index) հնարավորություն է տալիս կատարել արդյունավետ մասնակի համընկնման որոնում՝ առանց բոլոր փաստաթղթերում կանոնավոր արտահայտությունների (regular expression) սկանավորման պահանջի:

1.3. Անվտանգության ապահովում, վավերացում և սխալների կենտրոնացված մշակում

IronPulse-ում վավերացումն (Authentication) իրականացվում է **JSON Web Tokens (JWT)**-ի միջոցով: Երբ օգտատերը գրանցվում կամ հաջողությամբ մուտք է գործում համակարգ, սերվերը գեներացնում է ստորագրված JWT, որը որպես օգտակար բեռ (payload) պարունակում է օգտատիրոջ MongoDB փաստաթղթի ID-ն: Այս թոքենը վերադարձվում է հաճախորդին (client) և պետք է ներառվի պաշտպանված վերջնակետին (endpoint) ուղղված յուրաքանչյուր հաջորդ հարցման *Authorization* հեղերում (header)՝ օգտագործելով Bearer թոքենի սխեման: `auth` middleware-ը դուրս է բերում թոքենը հեղերից, ստուգում է դրա ստորագրությունը միջավայրի փոփոխականներում (environment variables) պահվող JWT գաղտնիքի (secret) համեմատ, վերծանում է payload-ը՝ օգտատիրոջ ID-ն ստանալու համար, հարցում է անում տվյալների բազային՝ հաստատելու, որ օգտատերը դեռ գոյություն ունի, և կցում է օգտատիրոջ ամբողջական փաստաթուղթը հարցման (request) օբյեկտին՝ հաջորդող controller-ների կողմից օգտագործման համար: Եթե թոքենը բացակայում է, սխալ է ձևավորված կամ ժամկետանց է, middleware-ն անմիջապես վերադարձնում է 401 *Unauthorized* պատասխան՝ թույլ չտալով հարցման հետագա ընթացքը:

Գաղտնաբառերի անվտանգությունը կառավարվում է **bcryptjs** գրադարանի միջոցով, որն իրականացնում է `bcrypt` հեշավորման ալգորիթմը: Երբ նոր օգտատեր է գրանցվում, բաց տեքստով գաղտնաբառը երբեք չի պահվում: Փոխարենը, `bcrypt`-ը գեներացնում է պատահական աղ (salt) և համատեղում այն գաղտնաբառի հետ՝ ստանալով ֆիքսված երկարության հեշ (hash), որը հաշվողական առումով անհնար է հետ շրջել: Մոլթի ռաունդների (salt rounds) քանակը կարգավորելի է միջավայրի փոփոխականների միջոցով (լռելյայն սահմանված է 12), ինչն ապահովում է ամուր հավասարակշռություն անվտանգության և սերվերի արդյունավետության միջև: Մուտք գործելիս `bcryptjs`-ը համեմատում է ներկայացված բաց տեքստով գաղտնաբառը պահված հեշի հետ՝ օգտագործելով հաստատուն ժամանակի համեմատության ֆունկցիա (constant-time comparison), ինչը կանխում է ժամանակային գրոհները (timing attacks): User սխեման օգտագործում է նաև Mongoose-ի pre-save հուկ (hook)՝ գաղտնաբառն ավտոմատ կերպով

հեշավորելու համար ամեն անգամ, երբ այն փոփոխվում է՝ երաշխավորելով, որ հուս գաղտնաբառը երբեք չի դիպչում տվյալների բազայի շերտին:

Մուտքային տվյալների ստուգումը (validation) կատարվում է **express-validator** գրադարանի միջոցով, որը ներառում է հանրաճանաչ validator.js գրադարանը Express-ին համատեղելի middleware-ի մեջ: Վավերացման կանոնները սահմանվում են դեկլարատիվ կերպով middleware շերտում՝ առանձին controller-ի տրամաբանությունից: Գրանցման երթուղու (route) համար կանոնները պահանջում են, որ օգտատիրոջ անունը լինի 3-ից 30 նիշի սահմաններում, էլ. փոստը լինի վավեր ձևաչափով և բերվի ստորին տառատեսակի (lowercase), գաղտնաբառը լինի առնվազն 6 նիշ, իսկ ցանկացած կամընտիր դաշտ, ինչպիսիք են տարիքը, քաշը և հասակը, պարունակի դրական թվեր իրատեսական միջակայքերում: Մարզումների ստեղծման համար կանոնները պահանջում են, որ անվանումը առկա լինի և չանցնի 100 նիշը, կատեգորիան համապատասխանի նախապես սահմանված enum արժեքներից մեկին, տևողությունը լինի առնվազն 1 րոպե, իսկ այրված կալորիաները լինեն 0 կամ ավելի: Ընդհանուր օգտագործման validate օժանդակ ֆունկցիան ստուգում է բոլոր կուտակված կանոնների արդյունքը, և եթե որևէ մեկը ձախողվում է, անմիջապես վերադարձնում է 400 Bad Request պատասխան, որը պարունակում է սխալների օբյեկտների կառուցվածքային զանգված (որտեղ նշվում է դաշտը և մարդու համար ընթերցելի հաղորդագրությունը)՝ նախքան controller-ի կանչվելը:

Սխալների մշակումը (Error handling) կենտրոնացված է հատուկ middleware-ում, որը գտնվում է Express middleware շղթայի վերջում և որսում է յուրաքանչյուր սխալ, որը փոխանցվում է next ֆունկցիայի միջոցով: Այս մշակիչը նույնականացնում է սխալի տեսակը և այն համապատասխանեցնում HTTP կարգավիճակի կոդին ու հաղորդագրությանը: Mongoose CastError-ները, որոնք առաջանում են, երբ URL պարամետրում տրամադրվում է անվավեր MongoDB ObjectId, վերածվում են 404 պատասխանների՝ հստակ հաղորդագրությամբ: Mongoose-ի կրկնվող բանալու սխալները (duplicate key errors), որոնք առաջանում են, երբ օգտատերը փորձում է գրանցվել արդեն զբաղված էլ. փոստով կամ օգտատիրոջ անունով, վերածվում են 409 Conflict պատասխանների, որոնք նշում են հակասող դաշտի անունը: Mongoose վավերացման սխալները, որոնք առաջանում են, երբ

տվյալները չեն համապատասխանում սխեմայի մակարդակի սահմանափակումներին, վերածվում են 400 պատասխանների՝ խախտված կանոնների ամբողջական ցանկով: JWT-ին բնորոշ սխալները վերածվում են 401 պատասխանների: Մշակման (development) ռեժիմում սխալի ամբողջական հետագիծը (stack trace) ներառվում է պատասխանի մեջ՝ կոդի սրբագրմանը (debugging) օգնելու համար: Արտադրական (production) ռեժիմում վերադարձվում են միայն կարգավիճակի կոդը և հաղորդագրությունը:

ԳԼՈՒԽ 2. API ՎԵՐՋՆԱԿԵՏԵՐԻ ՖՈՒՆԿՑԻՈՆԱԼՈՒԹՅՈՒՆԸ ԵՎ FRONTEND ԻՐԱԿԱՆԱՑՈՒՄԸ

2.1. Մարզումների վերջնակետերի գործառնությունները և սերվերային վերլուծությունը (Analytics)

Մարզումների վերջնակետերը (endpoints) ներառում են մի քանի գործառնություններ, որոնք API-ն գործնական են դարձնում իրական կիրառման համար: Ցուցակի (list) վերջնակետն ապահովում է էջավորում (pagination)՝ `page` և `limit` հարցման պարամետրերի միջոցով, ընդ որում սահմանաչափը (limit) սահմանափակված է առավելագույնը 50 արդյունքով՝ չարաշահումները կանխելու համար: Այն աջակցում է ազատ տեքստային որոնում մարզման անվան դաշտում՝ օգտագործելով մեծատառ-փոքրատառ հաշվի չառնող (case-insensitive) կանոնավոր արտահայտություն: Այն աջակցում է գտում ըստ կատեգորիայի ճշգրիտ արժեքի: Այն աջակցում է տեսակավորում ըստ ամսաթվի՝ աճման կամ նվազման կարգով: Պատասխանը ներառում է էջավորման մետատվյալների օբյեկտ, որը պարունակում է ընթացիկ էջը, էջերի ընդհանուր քանակը, մարզումների ընդհանուր քանակը, սահմանաչափը և բուլյան (boolean) դրոշներ այն մասին, թե արդյոք գոյություն ունի հաջորդ կամ նախորդ էջ: Մարզումների յուրաքանչյուր հարցում պարտադրում է օգտատիրոջ սեփականության իրավունքը՝ ներառելով վավերացված օգտատիրոջ ID-ն որպես գտման պայման, ինչն անհնար է դարձնում մի օգտատիրոջ համար մեկ այլ օգտատիրոջ մարզումներին հասանելիություն ստանալը, դրանք փոփոխելը կամ ջնջելը՝ անկախ նրանից, թե նրանք գիտեն մարզման ID-ն, թե ոչ:

Առաջընթացի (progress) վերջնակետերը օգտագործում են MongoDB-ի ագրեգացման համակարգը (aggregation framework)՝ վիճակագրությունը սերվերի կողմում հաշվարկելու համար, այլ ոչ թե բոլոր փաստաթղթերը հավելվածի հիշողության մեջ բեռնելու միջոցով: Ամփոփման (summary) վերջնակետը գործարկում է երկու զուգահեռ ագրեգացման խողովակաշարեր (aggregation pipelines): Առաջինը խմբավորում է օգտատիրոջ բոլոր մարզումները մեկ փաստաթղթի մեջ, որը հանրագումարի է բերում մարզումների քանակը, գումարում է ընդհանուր տևողությունը և այրված կալորիաների ընդհանուր քանակը,

ինչպես նաև հաշվում է մեկ պարապմունքի միջին տևողությունն ու կալորիաները: Երկրորդը խմբավորում է մարզումները ըստ կատեգորիաների՝ ստեղծելով վարկանիշային ցուցակ, թե քանի պարապմունք, ընդհանուր բույն և ընդհանուր կալորիա է բաժին ընկնում յուրաքանչյուր կատեգորիային: Շաբաթական (weekly) վերջնակետը հաշվարկում է անցած 7 օրն ընդգրկող ամսաթվերի միջակայքը և այդ միջակայքում գտնվող մարզումները խմբավորում է ըստ իրենց ամսաթվի տողի՝ ստանալով օրական քանակը, տևողությունը և կալորիաների հանրագումարները: Այնուհետև հավելվածը կառուցում է ամբողջական յոթօրյա քարտեզ՝ սկզբնավորված զրոներով, և դրա մեջ է միավորում տվյալների բազայի իրական արդյունքները՝ երաշխավորելով, որ ակտիվություն չունեցող օրերը դեռևս հայտնվում են պատասխանի մեջ, այլ ոչ թե լռելյայն բաց թողնվում:

2.2. Թեթև և առանց ֆրեյմվորքի առաջնամասի (Frontend) իրականացումը

Առաջնամասը (frontend) մեկ ինքնաբավ HTML ֆայլ է, որի չափը մոտավորապես 70 կիլոբայթ է: Այն չի օգտագործում արտաքին JavaScript ֆրեյմվորք, կողի հավաքման համակարգ, մոդուլների հավաքող (module bundler) կամ փաթեթների կառավարիչ: Ամբողջ ինտերֆեյսը՝ ներառյալ էջերի միջև երթուղավորումը, API հաղորդակցությունը, ձևաթղթերի (forms) մշակումը, մոդալ պատուհանների կառավարումը, դիագրամների պատկերումը, թոստ-ծանուցումները (toast notifications) և էջավորումը, իրականացված է մաքուր JavaScript-ով (plain JavaScript): Վիզուալ ձևավորման մեջ օգտագործվում են Google-ի երկու անհատական տառատեսակներ՝ **Bebas Neue**-ն՝ սեղմված, միայն մեծատառերով ցուցադրման տառատեսակ, որն օգտագործվում է վերնագրերի և լոգոյի համար, և **DM Sans**-ը՝ երկրաչափական հումանիստական sans-serif տառատեսակ, որն օգտագործվում է հիմնական տեքստի և UI պիտակների համար: Գունային պնակը խարսխված է մուգ ածխագույն ֆոնով (#0E0E0E հեքս կոդով), քարտերի և պանելների համար նախատեսված մի փոքր ավելի բաց մակերեսային գույնով, և վառ կիտրոնագույն-կանաչով (#C8F135), որն օգտագործվում է հիմնական գործողությունների, ակտիվ վիճակների և տվյալների ընդգծման համար: Այս համադրությունը ստեղծում է ուժեղ վիզուալ հակադրություն (contrast)՝ պահպանելով ֆիթնեսին ուղղված, էներգետիկ էսթետիկան:

Հաճախորդի կողմից երթուղավորումը (Client-side routing) կատարվում է `navigate` ֆունկցիայի միջոցով, որը ցուցադրում և թաքցնում է էջի բաժինները CSS դասերի (classes) միացման/անջատման միջոցով, թարմացնում է կողային նավիգացիայի տարրերի ակտիվ վիճակը և գործարկում է յուրաքանչյուր էջի տվյալների բեռնման համապատասխան ֆունկցիան: Վավերացման վիճակը պահպանվում է `localStorage`-ում (թե՛ JWT թոքենը, թե՛ սերիալիզացված օգտատիրոջ օբյեկտը), այնպես որ սեսիաները պահպանվում են էջի թարմացումից հետո՝ առանց օգտատիրոջից կրկին մուտք գործելու պահանջի: API-ի հիմնական URL-ը նույնպես պահվում է `localStorage`-ում և կարող է փոփոխվել աշխատանքի ընթացքում (runtime)՝ թե՛ վավերացման էկրանին, թե՛ համակարգ մուտք գործած ինտերֆեյսի վերին տողում երևացող կարգավորումների դաշտից, ինչը հեշտացնում է frontend-ը տարբեր backend միջավայրերին ուղղորդելը:

Շաբաթական սյունակավոր դիագրամը (bar chart) և առաջընթացի կալորիականության դիագրամը պատկերվում են մաքուր HTML-ի և CSS-ի միջոցով՝ առանց որևէ դիագրամների գրադարանի: Յուրաքանչյուր սյունակ `div` տարր է, որի բարձրությունը հաշվարկվում է որպես տվյալների բազայի առավելագույն արժեքի տոկոս, իսկ CSS անցումները (transitions) անիմացնում են սյունակները բեռնման ժամանակ: Կատեգորիաների վերլուծությունը օգտագործում է նմանատիպ մոտեցում. յուրաքանչյուր տող պարունակում է հետագծի `div`, որի ներսում լրացված `div`-ի լայնության տոկոսը ներկայացնում է տվյալ կատեգորիայի մասնաբաժինը ընդհանուր մարզումներում: Այս թեթև իրականացումները խուսափում են դիագրամների ամբողջական գրադարան ներմուծելու ծախսերից այնպիսի պատկերումների համար, որոնք ի վերջո պարզ վիզուալիզացիաներ են:

ԵԶՐԱԿԱՑՈՒԹՅՈՒՆ

IronPulse-ը ներկայացնում է ամբողջական, արտադրությանն ուղղված մոտեցում ֆիթնեսի հետևման հավելված կառուցելու համար՝ օգտագործելով բացառապես բաց կոդով, անվճար հասանելի տեխնոլոգիաներ: Նախագիծը ցույց է տալիս, որ լավ կառուցված Express API-ն, երբ համակցված է Mongoose սխեմաների մտածված օգտագործման, JWT վավերացման, bcrypt գաղտնաբառերի անվտանգության, դեկլարատիվ մուտքային տվյալների ստուգման և ագրեգացման վրա հիմնված վերլուծության հետ, կարող է ապահովել backend, որը միաժամանակ անվտանգ է, արդյունավետ և հեշտ ընկալելի: Backend-ի յուրաքանչյուր շերտ ունի մեկ, հստակ սահմանված պատասխանատվություն, և այդ շերտերի միջև առանձնացումը նշանակում է, որ ցանկացած անհատական բաղադրիչ՝ վավերացման կանոնները, սխալների մշակիչը, authentication middleware-ը, առաջընթացի ագրեգացման տրամաբանությունը, կարող է փոփոխվել, փոխարինվել կամ ընդլայնվել՝ առանց մյուսները խաթարելու:

Frontend-ը դեմոնստրացնում է, որ բարդ օգտատիրոջ ինտերֆեյսները չեն պահանջում ծանրակշիռ ֆրեյմվորքներ կամ կոդի հավաքման բարդ խողովակաշարեր (pipelines): Մեկ HTML ֆայլը, CSS ճարտարապետությանը պատշաճ ուշադրություն դարձնելու, JavaScript-ի մտածված կազմակերպման և ուժեղ վիզուալ ձևավորման ուղղության շնորհիվ, կարող է ստեղծել հավելված, որն իրեն զգալ է տալիս հղկված, արագագործ և իսկապես հաճելի օգտագործման համար: Frontend ֆրեյմվորքից խուսափելու որոշումը նաև նշանակում է, որ հաճախորդի կոդմիջ կառավարվող զրոյական կախվածություններ (dependencies) կան, չկան տարբերակների կոնֆլիկտներ, հավաքման արտեֆակտներ (build artifacts) և ոչ մի գործիքակազմ չի պահանջվում՝ բացի վեբ բրաուզերից:

Ֆիթնեսի տեսանկյունից հավելվածն ընդգրկում է մարզումային կենսակերպի ամբողջական ցիկլը՝ մարզումների պլանավորում և գրանցում, ժամանակի ընթացքում առաջընթացի հետևում, հասկացողություն այն մասին, թե ինչպես են տարբեր կատեգորիայի վարժությունները նպաստում ընդհանուր ակտիվությանը, և սննդային ուղեցույցների ստացում, որոնք լրացնում են ֆիզիկական մարզումները: Շաբաթական վիճակագրությունը օգտատերերին տալիս է գործնական արձագանք իրենց հետևողականության վերաբերյալ,

մինչդեռ ամբողջ ժամանակահատվածի հանրագումարները ապահովում են մոտիվացիոն բավարարվածություն՝ տեսնելով, թե ինչպես է կուտակված ջանքը վերածվում մեծ թվերի:

Նախագծի հետագա զարգացման ուղղությունները կարող են ներառել նպատակների սահմանման գործառույթների ավելացում, որտեղ օգտատերերը սահմանում են շաբաթական թիրախային տևողություններ կամ կալորիաների այրում և հետևում են իրենց առաջընթացին այդ նպատակների համեմատ, սոցիալական շերտի ներդրում, որտեղ օգտատերերը կարող են կամընտիր կերպով կիսվել մարզումների ամփոփագրերով, մարզումների կադապարների (templates) աջակցության ավելացում, որոնք կարող են վերաօգտագործվել տարբեր պարապմունքների ընթացքում, ինտեգրում կրելի սարքերի (wearable devices) API-ների հետ՝ մարզումների տվյալները ավտոմատ ներմուծելու համար, և բնիկ մոբիլ հաճախորդների (native mobile clients) կառուցում React Native-ի կամ Flutter-ի միջոցով, որոնք կօգտագործեն նույն REST API-ն՝ առանց backend-ի որևէ փոփոխության: API-ն նախագծվել է այս ընդլայնումները հաշվի առնելով. վավերացման, տվյալների և վերլուծության վերջնակետերի հստակ առանձնացումը նշանակում է, որ նոր գործառույթները կարող են ավելացվել որպես նոր երթուղային խմբեր (route groups)՝ առանց գոյություն ունեցող կոդին դիպչելու:

Ի վերջո, IronPulse-ը ծառայում է և՛ որպես գործնական ֆիթնես գործիք, և՛ որպես ժամանակակից JavaScript վեբ մշակման պրակտիկայի տեղեկատու իրականացում (reference implementation)՝ ցույց տալով, թե ինչպես գրադարանների փոքր, բայց լավ ընտրված հավաքածուն և կարգապահ ճարտարապետական մոտեցումը կարող են արտադրել ծրագրային ապահովում, որը շատ ավելի մեծ է, քան իր առանձին մասերի գումարը:

Օգտագործված գրականության ցանկ

- **Node.js:** Node.js Foundation. *Node.js — JavaScript Runtime*. Available at: <https://nodejs.org>. (Սերվերային միջավայր backend-ի և իրադարձությունների վրա հիմնված I/O մոդելի գործարկման համար):
- **Express.js:** OpenJS Foundation. *Express.js — Web Framework for Node.js*. Available at: <https://expressjs.com>. (Վեբ ֆրեյմվորք HTTP երթուղավորման և middleware շղթայի կառուցման համար):
- **MongoDB:** MongoDB, Inc. *MongoDB — The Developer Data Platform*. Available at: <https://www.mongodb.com>. (Փաստաթղթերի վրա հիմնված NoSQL տվյալների բազա օգտատերերի և մարզումների տվյալների պահպանման համար):
- **Mongoose:** Mongoose Contributors. *Mongoose — MongoDB Object Modeling for Node.js*. Available at: <https://mongoosejs.com>. (Օբյեկտային մոդելավորման գրադարան տվյալների բազայի սխեմաների և վավերացման կանոնների սահմանման համար):
- **Bcrypt Ալգորիթմ:** Provos, N. and Mazières, D. *A Future-Adaptable Password Scheme*. USENIX, 1999. (Գիտական թեզ, որը հանդիսանում է գաղտնաբառերի անվտանգ հեշավորման տեսական հիմքը):
- **Bcryptjs:** npm: bcryptjs. *Optimized bcrypt in JavaScript*. Available at: <https://www.npmjs.com/package/bcryptjs>. (Մաքուր JavaScript-ով գրված գրադարան գաղտնաբառերի ապահով հեշավորման և ստուգման համար):
- **JWT Ստանդարտ:** Auth0. *Introduction to JSON Web Tokens*. Available at: <https://jwt.io/introduction>. (JWT (RFC 7519) ստանդարտի պաշտոնական ուղեցույցը առանց սեսիաների (stateless) վավերացման համար):
- **Jsonwebtoken:** npm: jsonwebtoken. *JSON Web Token implementation for Node.js*. Available at: <https://www.npmjs.com/package/jsonwebtoken>. (Գրադարան սերվերում JWT թոքենների ստորագրման և վավերացման համար):
- **Dotenv:** Motdotla. *dotenv — Loads Environment Variables from .env*. Available at: <https://github.com/motdotla/dotenv>. (Գործիք .env ֆայլից միջավայրի փոփոխականներն ու գաղտնի բանալիները process.env բեռնելու համար):

- **CORS:** npm: cors. *Node.js CORS Middleware*. Available at: <https://www.npmjs.com/package/cors>. (Middleware հավելվածում Cross-Origin Resource Sharing հեղերների ավելացման և դիտարկիչի արգելքների շրջանցման համար):
- **Morgan:** Expressjs. *morgan — HTTP Request Logger Middleware for Node.js*. Available at: <https://github.com/expressjs/morgan>. (Օժանդակ գործիք մշակման փուլում HTTP հարցումների ավտոմատ լոգավորման և trace անելու համար):
- **Express-validator:** express-validator Contributors. *express-validator for Validator.js*. Available at: <https://express-validator.github.io>. (Դեկլարատիվ middleware մուտքային տվյալների ստուգման և սանիտարիզացիայի համար):
- **Bebas Neue Տառատեսակ:** Google Fonts. *Bebas Neue — Google Fonts*. Available at: <https://fonts.google.com/specimen/Bebas+Neue>. (Կոմպակտ տառատեսակ առաջնամասի (frontend) լոգոյի և գլխավոր վերնագրերի ձևավորման համար):
- **DM Sans Տառատեսակ:** Google Fonts. *DM Sans — Google Fonts*. Available at: <https://fonts.google.com/specimen/DM+Sans>. (Երկրաչափական sans-serif տառատեսակ հավելվածի հիմնական տեքստերի և UI պիտակների համար):
- **REST ճարտարապետություն:** Fielding, R. T. *Architectural Styles and the Design of Network-based Software Architectures*. Dissertation, 2000. (Հիմնարար աշխատություն, որը սահմանում է REST API-ների նախագծման սկզբունքները):
- **HTTP Արձանագրություն:** Mozilla Developer Network. *HTTP — MDN Web Docs*. Available at: <https://developer.mozilla.org/en-US/docs/Web/HTTP>. (Տեղեկատու փաստաթղթեր HTTP մեթոդների և կարգավիճակի կոդերի (200, 400, 401, 404) ճիշտ կիրառման համար):